

The Role of a Project-Based Capstone Course

Yael Dubinsky

Department of Computer Science

Technion – Israel Institute of Technology

yael@cs.technion.ac.il

Orit Hazzan

Department of Education in Technology and Science

Technion – Israel Institute of Technology

oritha@tx.technion.ac.il

ABSTRACT

A project-based capstone course aims at using software development skills while performing a project in the course domain. One of our main challenges is to simulate a real world environment so to provide our students with the experience they need. Planning this experience we should consider academic constraints as well as the students' schedule and skills. In this paper we describe how we implement an agile software development method in a project-based capstone course in the domain of operating systems. We elaborate on how we simulate a real world environment and present a role scheme that is used by the students to manage the process. We suggest a discussion on how to use the role scheme as an assessment tool to measure the development process in general and students' contribution in particular. We expect to extend and refine the comprehension regarding process measurement in students' teams at the academia.

Categories and Subject Descriptors

D.2.9 [Management]: Software process models, Programming teams, Productivity. D.2.8 [Metrics]: Process metrics. K.3.2 [Computer and Information Science Education]: Computer science education.

General Terms

Management, Measurement, Human Factors.

Keywords

Roles scheme, project metrics, project-based course.

1. INTRODUCTION

Since Summer 2002 we use Extreme Programming (XP) [1] as the development method for the operating systems projects developed as part of the 'Projects in Operating Systems' course taught at the Department of Computer Science at Technion. A team of four academic coaches guide the students in both operating systems and software engineering domains. Till today 27 such projects were performed by about 300 students in 8

semesters. During this period we have built and constantly refine a teaching and coaching framework to support the course.

In addition to the introduction of XP into the course, some studio elements were also introduced to support learning processes. The Studio is the basic learning method used in architecture schools and it has been adopted as a training environment by some other design disciplines. In such studios, students, guided closely by tutors, develop projects and are engaged in an on-going reflection process, both on what they are creating and on the creation process itself. An XP studio includes computers, tables and white boards, and is used by 12 students and one academic coach for the entire semester, only for the project development purposes. This way the Computer Science Department at the Technion has successfully managed to provide a simulation of a real world environment for the sake of the course students. The combination of the use of the studio and XP in the course creates a conceptually-physically stimulating software development environment. It turns out that although the format of the 'Operating Systems Projects' course includes also the teaching of a software development method, the operating systems-related material traditionally taught in the course was not changed or reduced. Rather, the introduction of XP into the course was found to enrich the development environments with respect to topics such as customer needs and testing issues.

In [2] we suggest and detailed a role scheme to be used in software development teams. This role scheme is based on the idea that software development is a complicated task and so does its management. Therefore, the role scheme suggests a management distribution among teammates in a way that every teammate plays the role of a developer as well as a special role which is part of the project management. This way we have many aspects that are managed in parallel and we gain accountability among all teammates. We have defined four groups of roles. The first is the leading group which consists of the roles as a coach and a tracker as defined in XP. The second is the customer group which consists of the roles of the customer, the end user, and the in charge of acceptance tests. The third group is the code group which consists of the roles of in charge of design, of continuous integration, of unit testing, of algorithms and effectiveness, and of code control. The fourth last group is the maintenance group which consists of the roles of in charge of documentation, of presentations, and of installation shield. Using this role scheme we have a representative for each of the main practices of the software development process who is involved and most accountable of this practice. Moreover, every student gain managing skills and experience an extensive team work.

2. MEASUREMENT AND ASSESMENT

Implementing the role scheme we have defined a grading policy that grants 65% for the group part of grade and 35% for the personal part. The group part consists of meeting the requirements and time tables in both releases of the project (there are two releases with one iteration in the first release and two in the second release). It also consists of the project documentation and the group evaluation of academic coach. The personal part consists of the role performance, the test-first exercise, the presence in the project meetings, and the personal evaluation of academic coach.

There exist measurement quantitative approaches which support the management of software projects ([3], [4]). Using the grading policy, our goal is to refine it by deriving quantitative metrics during development. We have defined 3 main measures. The first is the Role Time Measure (RTM). It measures the ratio development/role-performance; that is, the time invested in development tasks relative to the time invested in role activities. The second measure is the Role Communication Measure (RCM) that measures the level of communication in the team at each development stage. This measure evolves since each role holder needs to communicate with the other team members in order to perform the individual role more efficiently. The third measure is the Role Management Measure (RMM) that measures the level of the project management. Since the role scheme aim at covering all management aspects, maximal level is obtained when all role holders provide maximum role performance.

An example of data that is gathered from an electronic forum of one of the course teams shows averaged RTM ratio of 70%(development)-30%(role), and highest level (49.2%) of RCM in the leading group in contrast with the lowest level (6.5%) of RCM in the customer group. Note that this data is of one project in one course and does not imply on other projects.

Due to space limitations we cannot provide with full quantitative data for measures illustration; still the measures and some data to

support them are our discussion object. We are in the process of refining these measures and we expect collaboration of theoretical and practical ideas.

3. SUMMARY AND DISCUSSION

In this paper we present an implementation of an agile software development method in a project-based capstone course. We provide the students with hands-on experience in team work and in management aspects. Using XP values and practices together with studio elements we create a real world environment to run the project. XP encourages social interactions while tight planning of time and resources is being done. We implement a role scheme that fits XP spirit, increase accountability of teammates, and simplify the project management. We have defined measures to control the process.

We would like to raise a discussion in the matter of the significance of projects in general and the role scheme in particular as an experiencing framework for teams in the academia.

4. REFERENCES

- [1] Beck K., *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2000.
- [2] Dubinsky Y. and Hazzan O. (2004). Roles in agile software development teams", 5th International Conf. on eXtreme Programming and Agile Processes in Software Engineering.
- [3] Pulford, K. and Combelles, K.A. and Shirlaw, S. (1996). *A quantitative Approach to Software Management – The ami Handbook*, Addison-Wesley.
- [4] Putnam, L.H. and Myers, W. (1997). *Industrial Strength Software – Effective Management Using Measurement*, IEEE Computer Society Press.